

Window Handling in Playwright

Target Locators

Handling Browser Windows / Tabs

In Playwright, each browser tab or popup is represented by a Page object. You work with that Page directly instead of switching a global driver focus.

WHAT

A browser window or tab is represented by a Playwright Page.

WHY

Applications may open a new tab/window when a link, button or action is triggered.

HOW

Capture the new Page with `waitForEvent('popup')` or inspect all open pages with `browserContext.pages()`.

Window / Tab

The Playwright mental model

One tab / window

Think of every browser tab or popup as a separate Page object.

No window handle string

Playwright does not use Selenium-style getWindowHandle() / getWindowHandles() for normal browser-tab automation.

Direct interaction

Once you have the Page object, use normal Playwright locators on that page.

The Golden Circle

What • Why • How

WHAT

A Page object represents a browser tab or window.

WHY

Multiple Page objects let you work with multiple tabs/windows in the same browser context.

HOW

Wait for the popup/page event, store the returned Page, and interact with it directly.

More About Window Handling

Pages instead of window handles

Selenium concept

Selenium exposes alphanumeric window handles and requires switching the driver to the required handle.

Playwright concept

Playwright exposes Page objects. Keep the required Page reference and call its methods directly.

Example idea

`const pages = context.pages();` gives the currently open pages in that browser context.

Key idea

There is no `driver.switchTo().window(...)` equivalent for ordinary Playwright tab/window handling.

How to Handle a New Window

Popup pattern

STEP 1

Create a promise for the popup before triggering the action.

STEP 2

Perform the click or action that opens the new window/tab.

STEP 3

Await the popup and store it as a Page object.

Basic syntax

```
const popupPromise = page.waitForEvent('popup');
await page.getByRole('link', { name: 'Open' }).click();

const popup = await popupPromise;
await popup.waitForLoadState();
```

Why before the click?

The event listener must be ready before the action that creates the popup.

How to Handle Multiple Windows

Using BrowserContext pages

STEP 1

Get all open pages from the current browser context.

Get all pages

```
const pages = context.pages();  
console.log(pages.length);
```

STEP 2

Identify the required Page using its title, URL or another reliable property.

Find by title

```
for (const p of context.pages()) {  
  console.log(await p.title());  
}
```

STEP 3

Use the selected Page directly.

Interact with selected page

```
const targetPage = context.pages()[1];  
await targetPage.getByRole('button', { name: 'Submit' }).click();
```

Wait for the New Page

Recommended event handling

Popup from current page

Use `page.waitForEvent('popup')` when the new tab/window is created by the current page.

New page in context

Use `browserContext.waitForEvent('page')` when you want to observe a newly created Page at the context level.

Context-level pattern

```
const pagePromise = context.waitForEvent('page');
await page.getByRole('link', { name: 'Open' }).click();
const newPage = await pagePromise;
await newPage.waitForLoadState();
```

Closing Windows / Tabs

Page-level and browser-level close

Close one tab

Use `await page.close()` to close the current Page.

Close a specific tab

Keep its Page reference and call `await targetPage.close()`.

Close the browser

Use `await browser.close()` when the entire browser session should end.

Example

```
await popup.close();

// End the complete browser session
await browser.close();
```

Selenium comparison

`driver.close()` closes the current window and `driver.quit()` ends the WebDriver session. Playwright uses Page and Browser objects instead.

Selenium → Playwright

Concept mapping

Get current window handle	<code>driver.getWindowHandle();</code>	→	Use the current page reference: <code>page</code>
Get all window handles	<code>driver.getWindowHandles();</code>	→	Use <code>context.pages()</code>
Switch to window	<code>driver.switchTo().window(handle);</code>	→	Use the required Page object directly
Wait for new window	Implicit through handle collection	→	<code>page.waitForEvent('popup')</code> / <code>context.waitForEvent('page')</code>
Close current window	<code>driver.close();</code>	→	<code>await page.close()</code>
Close all browser windows	<code>driver.quit();</code>	→	<code>await browser.close()</code>

Classroom Activity

Window handling with Playwright

1. Load

<https://www.irctc.co.in/>

2. Action

Click the FLIGHTS link that opens the flight-booking page.

3. Capture the new page

Use `page.waitForEvent('popup')` before clicking the link.

4. Get the title

Use `await flightPage.title()` and verify the returned title.

5. Close the first tab

Keep the original Page reference and call `await page.close()` after the flight page has been captured.

Playwright pattern

```
const flightPromise = page.waitForEvent('popup');
await page.getByRole('link', { name: 'FLIGHTS' }).click();

const flightPage = await flightPromise;
await flightPage.waitForLoadState();
console.log(await flightPage.title());

await page.close();
```

Practical note

The exact locator and popup behavior can change when the application UI changes. Inspect the current page and use the locator that matches the live application.

Summary

Window handling in Playwright

01

A browser tab/window is represented by a Page object.

02

Playwright does not use Selenium-style window handle strings for normal tab handling.

03

Use `page.waitForEvent('popup')` when an action opens a popup.

04

Use `context.waitForEvent('page')` to observe a newly created Page at browser-context level.

05

Use `context.pages()` to get all currently open pages.

06

Keep the required Page reference and interact with it directly.

07

Use `page.close()` for one tab and `browser.close()` for the complete browser session.

Mind Map

Playwright Window Handling

Browser Context

Contains multiple Page objects

Page

Represents a browser tab / window

New popup

`page.waitForEvent('popup')`

New page

`context.waitForEvent('page')`

All pages

`context.pages()`

Switching

No `switchTo().window()` → use the required Page object

Close one

`page.close()`

Close browser

```
browser.close()
```